
INFS7202 Web Information Systems

GeoMetaAssist Design Document

AI-Powered Geospatial Metadata Editor

STAC-compliant metadata generation powered by RAG

Submitted by

Student: Dwiputra Sam Mulia

Student ID: s4980498

Course: INFS7202 — Web Information Systems

Instructor: Dr. Aneesha Bakharia

Date: April 14, 2026

Table of Contents

1	<i>Project Overview & Key Features</i>	1
1.1	Key Features	2
2	<i>Database Design</i>	3
2.1	Design Rationale of implementing BCNF instead of 3NF	4
2.2	Design Rationale for JSON as field types	4
3	<i>HTML Mockup</i>	6
4	<i>Technology Research</i>	9
4.1	Decision 1: Metadata Standard — STAC 1.0.0 vs ISO 19115.....	9
4.2	Decision 2: GeoJSON Metadata Extraction — Client-side (Turf.js) vs Server-side (GeoPandas)	10
5	<i>Accessibility</i>	12
5.1	WCAG 2.1 Overview	12
5.2	WCAG 2.1 Example of Criteria Applied to GeoMetaAssist	12
5.3	Cognitive Accessibility	14
5.4	Accessibility Testing Strategy	14
6	<i>Security</i>	15
6.1	Authentication.....	15
6.2	Authorisation and Role-Based Access Control (RBAC)	15
6.3	Object-Level Permission Enforcement	16
6.4	Subscription Quota Enforcement	17
6.5	CSRF Protection.....	17

6.6	Cross-Site Scripting (XSS) Prevention	17
6.7	File Upload Security	18
6.8	Security Headers and HTTPS	18
6.9	DEBUG Mode and Secret Key Management	19
<i>References</i>		<i>iv</i>
<i>Use of Generative AI and Other Tools</i>		<i>vi</i>

1 Project Overview & Key Features

GeoMetaAssist is a cloud based SaaS solution developed to automate the production of standardized geospatial metadata for spatial datasets. GeoMetaAssist was created to address an ongoing inefficiency in the geospatial community regarding the manual time consuming process of generating metadata records compliant with open catalogue standards. Specifically GeoMetaAssist supports the SpatioTemporal Asset Catalog (STAC) format as its target catalogue standard.

Problem Domain. The routine requirement for geospatial professionals is to have structured metadata accompany their dataset so it can be discovered, shared across different systems and comply with open data framework requirements. However, this manual process is very labour intensive, prone to errors and requires an intimate knowledge of the target catalogue schema. Current tools do provide little assistance to help guide the practitioner and also do not use artificial intelligence (AI) to assist the practitioner complete all the fields in the catalogue document. Therefore, geospatial practitioners will continue to need to read through large volumes of information contained within specification documents individually.

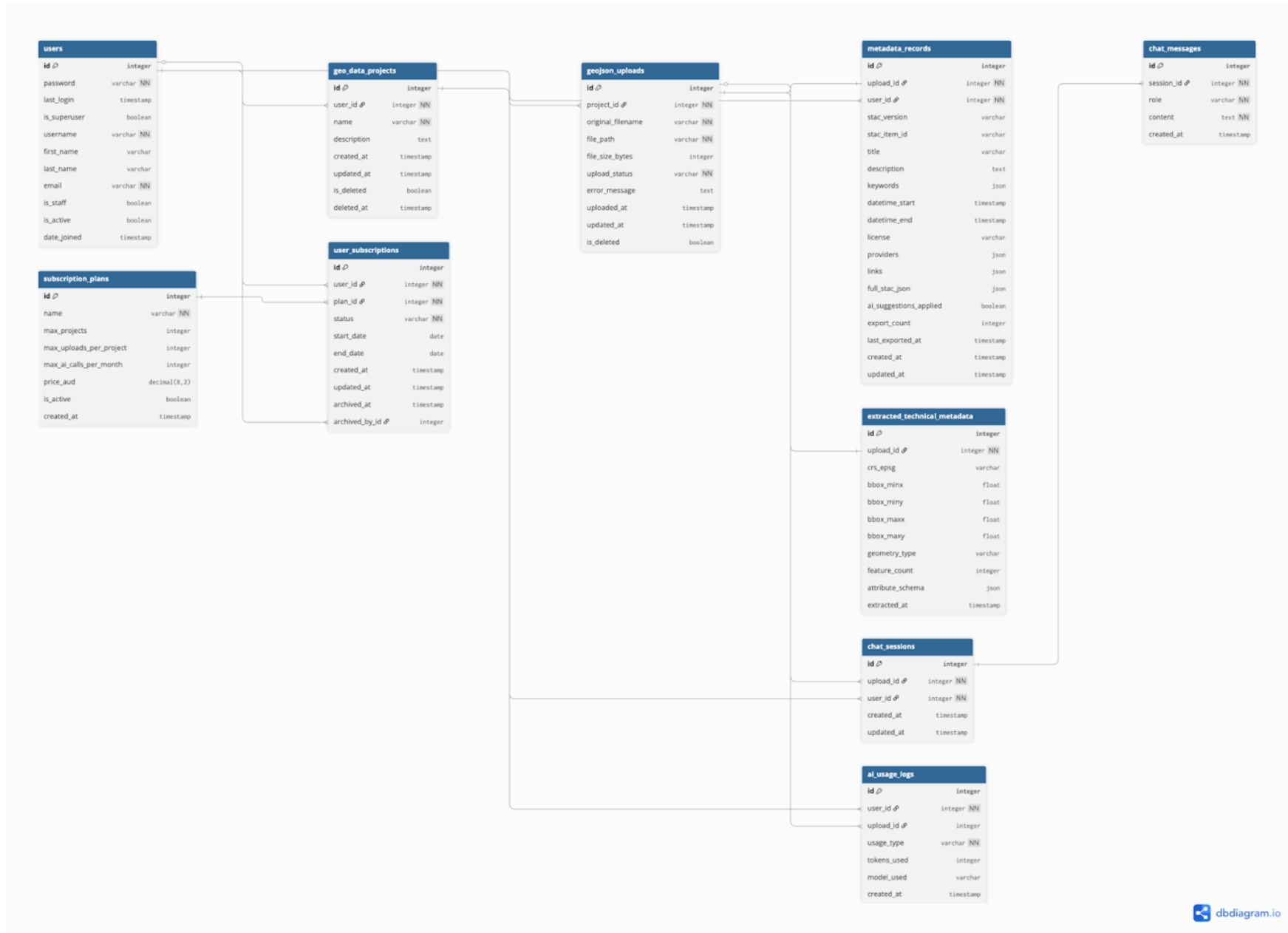
Target audience: The main users of GeoMetaAssist are geospatial data engineers, GIS analysts and those responsible for managing datasets prior to publishing them or archiving them. This also include Research Organizations and Government Agencies which are required to publish STAC compliant metadata, and Spatial Data Consultancy companies which produce metadata deliverable products for their client base.

Platform Purpose. GeoMetaAssist takes in GeoJSON Files as input, and outputs STAC item in JSON. GeoMetaAssist uses a combination of automated extraction of technical metadata field values using programmable methods and a retrieval-augmented generation (RAG) pipeline built on top of an index of STAC specification documentation to suggest descriptive metadata fields based upon the context of user's interaction with the system and assist users in navigating through the appropriate context-based fields by way of an interactive AI chat bot

1.1 Key Features

1. **User authentication and subscription management.** Users can enter their information in order to sign up and/or login to the application with an e-mail address and a password. A two-tiered software as a service (SaaS) is used; the free version offers a limited number of projects and has limitations on uploads and number of requests made to the AI portion. In addition, there is an administrator portal available for employees to create, update and archive a registered subscriber's subscription account.
2. **Project management.** Registered users have the ability to create, read, write and delete projects containing geo-data. These are considered units of organization for uploading one or more GeoJSON data sets along with related metadata.
3. **GeoJSON data set and technical metadata extraction.** Once uploaded into the platform, the geojson data sets will trigger the automatic extraction of the coordinate reference system (CRS/EPSG) , spatial bounding box, geometry type, feature count, and attribute schema; these extracted parameters populate the technical field of the STAC item record without requiring any manual entry.
4. **AI-assisted descriptive metadata generation.** Using RAG pipeline and operating on top of the STAC specification, the system will suggest values for descriptive fields (title, description, keywords) generated from the dataset's attribute names and geometry characteristics. Additionally, RAG also generates recommended values for fields dependent upon context such as temporal extent.
5. **Interactive Metadata Chatbot.** An AI-powered chatbot running on top of the STAC specification via RAG will guide users inserting fields that cannot be inferred programmatically, such as license, providers, and external links.
6. **Metadata review/edit/export.** In addition to reviewing all of the AI-generated and auto-extracted values in a structured edit interface prior to exporting the fully populated record as a valid STAC-item json file. All exported records remain in storage and can be viewed from within the project dashboard for potential editing and re-exporting.

2 Database Design



2.1 Design Explanation

This database is modelled with a system that make every user/subscriber will have only one subscription that is actively in use at any given time and that subscription will always belong to a specific plan level. Every user may create projects, each project may contain from one or many GeoJSON files. For each file uploaded into the platform it will generate both the technical metadata for the upload and the STAC metadata for the file. All metadata created by uploading files will be associated with a single chat session that contains all of the conversations between the users and the AI chatbot metadata assistant. Usage of the AI will be tracked on a per-user basis so that there are quotas enforced on subscriptions based on total number of RAGs and Chatbot interactions.

2.2 Design Rationale of implementing BCNF instead of 3NF

GeoMetaAssist's database schema is structured to follow the standards of Boyce Codd Normal Form (BCNF), as opposed to use the 3NF. Although 3NF removes transitive dependency and can meet the needs of most applications, BCNF represents a stronger assurance, specifically it assures that each determinant within each table will be a candidate key. Therefore, this ensures that no anomaly type (insert/update/delete) will exist due solely to structural issues in the schema, and establishes clear ownership boundaries between users, projects, uploads, and metadata records. Additionally, because multiple users access the same database in a SaaS model, and data isolation between subscribers must be implemented via the database schema, GeoMetaAssist has established these strong assurances through its use of a surrogate integer primary key in each table as the sole determinant of non-key attributes. Therefore, based upon structural characteristics alone, insert, update, and delete anomalies will never occur.

2.3 Design Rationale for JSON as field types

Some fields (attribute_schema, keywords, providers, links, and full_stac_json) have been defined using JSON. Due to the fact that there is no fixed number of attributes and their name and data type will be different for each and every GeoJSON file within the system, normalization on attribute_schema will neither be practical nor possible in the "traditional" sense.

All of the other JSON fields; keywords, providers, links, and full_stac_json were determined due to the use of the STAC specification. These are defined as arrays and/or nested objects in the STAC Item schema. The addition of relational table decomposition for these fields would introduce unnecessary join complexities. Also, no benefits can arise from this additional decomposition since the application does not read/write the JSON fields individually and never performs filtering/aggregation on individual sub-fields at the SQL level. Prior to writing the JSON fields to the database, all JSON fields are validated

at the application level to ensure data integrity. `Attribute_schema` is programmatically populated by GeoPandas with an assured structure. Keywords, Providers, Links are also validated against the STAC specification schema prior to saving the metadata record.

3 HTML Mockup

The full list of pages created can be seen on these files inside the repository:

#	Page	Template File	Route
1	Base layout (authenticated)	base.html	Layout only — no route
2	Base layout (public)	base_public.html	Layout only — no route
3	Landing page	landing.html	/
4	Login	login.html	/accounts/login/
5	Register	register.html	/accounts/register/
6	Dashboard	dashboard.html	/dashboard/
7	Create project	project_create.html	/projects/create/
8	Edit project	project_edit.html	/projects/<id>/edit/
9	Project detail	project_detail.html	/projects/<id>/
10	Upload GeoJSON	upload_geojson.html	/projects/<id>/upload/
11	Upload detail	upload_detail.html	/projects/<id>/uploads/<id>/
12	Metadata editor	metadata_editor.html	/projects/<id>/uploads/<id>/metadata/
13	Export success	export_success.html	/projects/<id>/uploads/<id>/export/
14	Admin subscriptions	admin_subscriptions.html	/admin-portal/subscriptions/
15	Admin subscription edit	admin_subscription_edit.html	/admin-portal/subscriptions/<id>/edit/

All pages for the mockup of this project is developed in Django Templates and Bootstrap CSS Framework Version 5.3.3

Below are several screenshot examples of the pages:

dashboard.html

GeoMetaAssist
STAC Metadata Editor

Dashboard

My Projects

Upload History

Settings

JD Jane Doe
Sign out

Dashboard

Pro Jane Doe

TOTAL PROJECTS

3

TOTAL UPLOADS

7

AI CALLS THIS MONTH

4 / 10

MY PROJECTS

+ New Project

Brisbane Flood Extent 2023

3 uploads

Flood extent mapping for the February 2023 event covering the Brisbane River catchment and surrounding suburbs.

Updated 2 days ago

View Edit

Sydney Coastal Erosion Monitor

3 uploads

Monitoring coastal erosion rates along the Northern Beaches from Manly to Palm Beach using LIDAR-derived shoreline positions.

Updated 1 week ago

View Edit

Melbourne Urban Heat Islands

1 upload

Mapping urban heat island intensity across metropolitan Melbourne using satellite-derived land surface temperature data.

Updated 3 weeks ago

View Edit

project_create.html

GeoMetaAssist
STAC Metadata Editor

Dashboard

My Projects

Upload History

Settings

JD Jane Doe
Sign out

Projects / Brisbane Flood Extent 2023

Pro Jane Doe

Brisbane Flood Extent 2023

Edit

Flood extent mapping for the February 2023 event covering the Brisbane River catchment and surrounding suburbs.

GEOJSON UPLOADS

Upload GeoJSON

Filename	Uploaded	Status	Actions
brisbane_flood_polygons.geojson	25 Mar 2026	Complete	View Metadata Delete
flood_depth_points.geojson	24 Mar 2026	Processing	View Metadata Delete
catchment_boundary_v2.geojson	20 Mar 2026	Error	View Metadata Delete

7

project_detail.html

GeoMetaAssist
STAC Metadata Editor

Dashboard

My Projects

Upload History

Settings

Jane Doe
Sign out

Projects / Brisbane Flood Extent 2023 / brisbane_flood_polygons.geojson

ProJane Doe

brisbane_flood_polygons.geojson

Uploaded 25 Mar 2026 · Status: Complete

Extracted Technical Metadata

CRS / EPSG	EPSG:4326
Bounding Box	152.88, -27.85, 153.55, -27.10
Geometry Type	Polygon
Feature Count	2,847
Attribute Schema	flood_depth_m (float) suburb_name (string) risk_level (string) area_ha (float)

AI Status

Step 1: Technical Extraction

CRS, bounding box, schema extracted successfully

Step 2: AI Suggestions Generated

Title, description, keywords, and license suggested

Step 3: Your Review

Review and edit the AI-suggested metadata

Review and Edit Metadata

metadata_editor.html

GeoMetaAssist
STAC Metadata Editor

Dashboard

My Projects

Upload History

Settings

Jane Doe
Sign out

Projects / Brisbane Flood Extent 2023 / brisbane_flood_polygons.geojson / Metadata Editor

ProJane Doe

Metadata Editor

STAC Metadata Fields

IDENTIFICATION

STAC Item ID

brisbane-flood-extent-2023-polygons

Title AI suggested

Brisbane Flood Extent Polygons — February 2023 Event

Description AI suggested

Polygon features representing the spatial extent of flooding along the Brisbane River catchment during the February 2023 flood event. Each feature includes modelled flood depth, suburb name, risk classification, and area in hectares.

Keywords AI suggested

flood × hydrology × Queensland ×

TEMPORAL

Start Datetime AI suggested

22/02/2023

End Datetime AI suggested

05/03/2023

RIGHTS AND PROVIDERS

License AI suggested

CC-BY-4.0

Providers

[{"name": "Queensland Government", "roles": ["producer", "host"]}]

JSON array of STAC provider objects

LINKS

Metadata Assistant

You

What license should I use for government flood data in Australia?

Assistant

Australian government datasets are commonly released under **CC-BY-4.0**, which requires attribution but allows commercial use. The STAC spec recommends using SPDX license identifiers. I've pre-selected CC-BY-4.0 for you — you can change it if your data has different terms.

You

Should I include temporal extent even though it's a one-off event?

Assistant

Ask about STAC metadata...

Powered by RAG over the STAC specification

4 Technology Research

GeoMetaAssist evaluates two key technology choices. Both are reviewed based on their ability to satisfy GeoMetaAssist's requirements and an appropriate selection based upon those evaluations is made at the end of this section.

4.1 Decision 1: Metadata Standard — STAC 1.0.0 vs ISO 19115

GeoMetaAssist's first evaluation involves the determination of the metadata standard to be used by GeoMetaAssist. The platform will support single metadata standard to be chosen as its output. A review of two standards has been conducted, which are ISO 19115, which is the international standard for geographic information metadata, and the SpatioTemporal Asset Catalog (STAC) Specification.

Criteria	STAC 1.0.0	ISO 19115
Output format	JSON (STAC Item)	XML (ISO 19139)
Web-native	Yes, designed for cloud and web APIs	No, XML-based and tooling-heavy
Complexity	Moderate. Core spec is concise and well-documented	High. Full standard spans hundreds of fields and multiple profiles
Tooling and ecosystem	Growing rapidly. Strong support in AWS, Google Earth Engine, Microsoft Planetary Computer	Mature but heavyweight. Common in government and enterprise GIS
Machine readability	Excellent. JSON is natively consumable by web apps and APIs	Need to parse XML format
Minimum required fields	Small, well-defined core set with optional extensions	Large mandatory field set requiring significant domain knowledge
Community adoption trend	Rapidly growing in modern cloud-native geospatial platforms	Stable but declining in cloud-native contexts

Suitability for SaaS web app	High. JSON output integrates directly with Django and frontend	Low. XML output adds unnecessary parsing overhead
-------------------------------------	--	---

Decision: STAC 1.0.0

Based on these reviews, STAC 1.0.0 was determined to be the best fit for use as the output from GeoMetaAssist. The native JSON format of STAC 1.0.0 fits well into the design structure of a Django-based Web Application. Furthermore, the concise nature of STAC 1.0.0 provides sufficient support for AI-Driven Field Completion. Finally, because many cloud geospatial platforms have adopted STAC 1.0.0, it is believed to represent the more relevant metadata standard for the intended user community of geospatial data engineers. In contrast, ISO 19115 provides additional complexity over what is required for a Web-Native SaaS delivery model.

4.2 Decision 2: GeoJSON Metadata Extraction — Client-side (Turf.js) vs Server-side (GeoPandas)

In order to determine which architecture would best suit GeoMetaAssist's needs, both client-side and server-side architectures approaches for extracting technical metadata from uploaded GeoJSON files (such as bounding box, CRS, geometry type, feature count, and attribute schema) were examined.

Criteria	Client-side (Turf.js)	Server-side (GeoPandas)
CRS support	None. Turf.js assumes WGS84 (EPSG:4326) only. Cannot detect or handle other coordinate systems.	Full. GeoPandas detects any EPSG code and supports reprojection.
File size handling	Limited. Large GeoJSON files (50MB+) risk browser memory crashes and slow UI.	Robust. Server handles arbitrarily large files independently of user device.

Attribute schema inference	Fragile. JavaScript loose typing makes precise data type detection unreliable.	Reliable. GeoPandas infers typed attribute schema (float, int, string) precisely.
Server compute cost	None. All computation is offloaded to the user browser.	Low to moderate. Scales with file size but manageable for a SaaS platform.
Data privacy at extraction stage	High. File never leaves the user machine during extraction.	Lower. File must be uploaded to the server before extraction begins.
Integration with backend	Poor. Extracted values require a separate API call to be sent back to the server.	Seamless. Extracted values are written directly to the database in the same pipeline.
RAG pipeline compatibility	Incompatible. The file must be on the server for the AI suggestion pipeline to process it.	Compatible. File is already on the server. Extraction and AI suggestion run in sequence.
Persistence of results	Requires an additional client-server round-trip to save extraction results.	Direct. Results are saved to the database immediately after extraction completes.

Decision: Server-side extraction with GeoPandas

GeoPandas is chosen for the extraction process in GeoMetaAssist, based on two limiting factors which prevent a viable client-side extraction solution from being used regardless of its benefits. First, because the GeoJSON file must be uploaded to the server in order to execute the RAG-based AI suggestion pipeline, it will be available at the server-side prior to any metadata extraction can begin anyway. Second, since the extracted technical metadata must be stored permanently in the database as a record associated with the uploaded file, executing the extraction from the server with GeoPandas will eliminate unnecessary client-server round trip and provide better CRS management capabilities, more reliable attribute schema type checking and no limitations on file size due to memory constraints on browsers.

5 Accessibility

GeoMetaAssist will follow the standards of WCAG 2.1 Level AA in its design to meet the Australian government's requirement [1] and the commonly used standard for all professional websites. Accessibility is treated as a design constraint throughout the development process rather than being added after completion.

5.1 WCAG 2.1 Overview

WCAG 2.1 provides guidelines on what accessibility means in terms of how accessible your site needs to be as defined by four guiding principles: Perceivable, Operable, Understandable, and Robust (POUR). The POUR principles each have Success Criteria that can be tested for conformance to three different levels of compliance with WCAG: A (minimum); AA (standard; this is where GeoMetaAssist will focus); and AAA (Enhanced). All accessibility barriers within a site will generally try to comply at least at Level AA [2].

5.2 WCAG 2.1 Example of Criteria Applied to GeoMetaAssist

Ten example criteria below are selected based on relevance to the platform's various UI components, such as the sidebar navigation, the GeoJSON upload zone, the multi-section metadata editor form, the AI chatbot panel, and the dynamic processing status indicators.

Criterion	Level	Example Application to GeoMetaAssist	Implementation
1.1.1 Non-text Content	A	Sidebar navigation uses icon-only links (Bootstrap Icons). These carry no visible text and must expose a text alternative to screen readers.	aria-label on all icon-only buttons and links.
1.3.1 Info and Relationships	A	The metadata editor groups fields into logical sections (Identification, Temporal, Rights). Structure must be conveyed programmatically, not by visual layout alone.	<fieldset> and <legend> wrapping each form group.

1.4.1 Use of Colour	A	Upload status badges (Complete, Processing, Error) and AI-suggested field indicators use colour as a primary differentiator. Colour alone is insufficient.	Text labels accompany all colour-coded badges. No information conveyed by colour only.
1.4.3 Contrast (Minimum)	AA	Primary colour pairing is dark green (#1a3c2e) on white (ffffff), yielding a contrast ratio of ~14.7:1. All custom colour pairs must meet the 4.5:1 minimum for normal text.	All design system colour pairs verified using WebAIM Contrast Checker.
1.4.10 Reflow	AA	The two-column metadata editor (form + chatbot) must reflow to a single column at 320px viewport width without horizontal scrolling.	Bootstrap 5 responsive grid collapses to single column below the md breakpoint. rem units throughout.
2.1.1 Keyboard	A	The drag-and-drop upload zone is mouse-only by default. All features must be operable without a pointer device, including the chatbot send action.	Standard <input type="file"> fallback provided. All interactive elements reachable and operable via Tab and Enter.
2.4.1 Bypass Blocks	A	The sidebar navigation repeats on every authenticated page. Keyboard users must be able to skip repeated navigation to reach the main content directly.	Skip-to-main-content link as the first focusable element on every authenticated page.
2.4.6 Headings and Labels	AA	All form fields in the metadata editor must have visible, descriptive labels. Placeholder text is not a substitute as it disappears on input and is not reliably announced by screen readers.	Explicit <label for="..."> elements on every form input. No input relies solely on placeholder text [3].
3.3.1 Error Identification	A	Upload failures (wrong format, size exceeded), metadata form validation errors, and AI pipeline errors must be	Inline error messages below the relevant field or component. Django form

		identified in text describing the specific issue.	validation errors passed to template context [4]
4.1.2 Name, Role, Value	A	The chatbot panel and AI processing status card update dynamically. These state changes must be communicated to assistive technologies without requiring a page reload.	aria-live="polite" on the chatbot message container and AI status card [5].

5.3 Cognitive Accessibility

Beyond what is required by formal WCAG guidelines, GeoMetaAssist will use principles of cognitive accessibility to reduce the level of user expertise needed to utilize this application by people who have no prior knowledge of the STAC Specification. To accomplish this, the metadata editor organizes similar or related fields together under specific section headers. Additionally, when requested by the user, the chatbot will provide contextually-based explanations of what each field represents, and the chatbot's AI-based suggested answers assist users to avoid having to remember all possible field format requirements. In addition, error messages within this system are provided as simple statements that identify both an error condition and how to take corrective actions. These methods align with the W3C suggestions regarding cognitive accessibility [6].

5.4 Accessibility Testing Strategy

The following methods will be used to verify compliance prior to submission:

- Automated scan for WCAG compliance such as colour contrast verification for all custom colour pairs using IBM Equal Access Accessibility Checker [7].
- Keyboard navigation manual test on all interactive components: sidebar, file upload, metadata form, chatbot, and export flow.

6 Security

GeoMetaAssist addresses two aspects of security, which are the standard web application security to protect both the user's data and the integrity of the application, and subscription-based access controlling the SaaS business logic. The two aspects are implemented at a framework-level via the use of built-in mechanisms for security as part of the Django Framework [8] plus explicit application layer controls that are not covered by the Django Framework.

6.1 Authentication

GeoMetaAssist authentication will rely on Django's built-in authentication framework. By default GeoMetaAssist will utilize Django's default password hasher PBKDF2 with SHA-256 and a random salt [9], hence plain text passwords are never stored or logged.

Additionally, following configs will be applied on top of Django's defaults:

- **Password validation:** AUTH_PASSWORD_VALIDATORS config will enforces minimum length (8 characters), rejects entirely numeric passwords, and blocks passwords too similar to the user's email address [9].
- **Secure session cookies:** SESSION_COOKIE_HTTPONLY = True config will prevents JavaScript from reading the session cookie, mitigating session theft via XSS. Furthermore, SESSION_COOKIE_SECURE = True config will ensure cookies are only transmitted over HTTPS [10].
- **Session invalidation:** Logout operation will call request.session.flush(), which will destroys the server-side session record rather than only clearing the client cookie [11].
- **Brute-force protection:** Django does not throttle login attempts by default, therefore django-axes will be configured to lock accounts after a set number of consecutive failed attempts, protecting against credential stuffing [12].

6.2 Authorisation and Role-Based Access Control (RBAC)

GeoMetaAssist implements three roles using Django's built-in user flags, enforced at the view layer [13].

- **Authenticated User** (is_staff=False): standard subscriber. Access is restricted to their own projects, uploads, and metadata records only.
- **Staff** (is_staff=True): platform administrator. Access to the subscription management portal. Cannot access other users' project or metadata data.
- **Superuser** (is_superuser=True): full Django admin access. Reserved for platform maintenance only.

The table below defines enforced permissions across all major resources:

Resource	Unauthenticated	Authenticated User	Staff (is_staff)	Superuser
Landing / Login / Register	Read	Redirect home	Redirect home	Redirect home
Dashboard	Deny (302)	Own data only	Own data only	All data
Geo Data Projects (CRUD)	Deny (302)	Own only	Own only	All
GeoJSON Uploads (CRUD)	Deny (302)	Own only	Own only	All
Metadata Editor + Export	Deny (302)	Own records	Own records	All
AI Chatbot (RAG)	Deny (302)	Quota-gated	Quota-gated	Unlimited
Subscription Management	Deny (302)	Deny (403)	Full access	Full access
Django Admin Panel	Deny (302)	Deny (302)	Limited	Full access

6.3 Object-Level Permission Enforcement

Django's `@login_required` decorator will check and confirms if a user is authenticated, but it would not prevent one user from accessing another user's resources by guessing a

URL (horizontal privilege escalation). GeoMetaAssist will address this issue by filtering every object retrieval by both primary key and the authenticated user's ID:

Example:

```
project = get_object_or_404(GeoDataProject, pk=pk, user=request.user)
```

A request for /projects/2/ from a user who does not own project 2 returns a 404, not a 403, to avoid confirming that the resource exists.

6.4 Subscription Quota Enforcement

Server-side validation will be performed to enforce free-tier usage limits at the view-layer, prior to creation of any resource or triggering of an AI pipeline. Additionally The ai_usage_logs table is queried at the start of every RAG or chatbot request to verify the monthly call count before proceeding.

6.5 CSRF Protection

By default, CsrfViewMiddleware config is enabled within Django. Consequently, All forms generated using Django templates include a csrf_token field embedded within the template. A token is issued during the session and validated upon receipt of all POST, PUT, patch, and delete requests. Any request submitted without a valid token is rejected with a http status code 403 [14].

6.6 Cross-Site Scripting (XSS) Prevention

As a result of its use of a template engine, Django automatically escapes all variables passed to it. As such, the user-supplied content cannot be interpreted as HTML or JavaScript [15], thus providing protection against XSS attack. The one area requiring explicit attention is the STAC JSON export preview rendered in a <pre> block, which must apply the escape filter explicitly even though the content is system-generated, as a defence-in-depth measure.

SQL Injection Prevention

the ORM APIs employed by Django by default construct parameterized queries at the driver level. No user supplied data should ever be concatenated or inserted directly into

a SQL string [15]. Raw SQL queries can still be executed in order to perform optimizations; however, they must always employ the parameterized format of `cursor.execute(SQL, [params])`, while ensuring that the user-input data does not contain any string formatting [16].

6.7 File Upload Security

The GeoJSON upload endpoint potentially presents risks such as malicious file content, or oversized files. In order to anticipate those risks, the following handlings will be applied:

- **MIME type and extension validation:** files are validated server-side against an allowlist (`application/geo+json`, `.geojson`). The client-supplied `Content-Type` header will not be trusted.
- **File size limit:** 50MB enforced at the view layer.
`DATA_UPLOAD_MAX_MEMORY_SIZE` config will be configured to reject oversized requests before they reach the view.

6.8 Security Headers and HTTPS

The following Django settings will be applied for the UQCloud Zone production deployment:

- `SECURE_SSL_REDIRECT = True` config which useful to redirects all HTTP requests to HTTPS [15].
- `SESSION_COOKIE_SECURE` and `CSRF_COOKIE_SECURE = True`, which will enforce cookies transmitted over HTTPS only [15].
- `X_FRAME_OPTIONS = "DENY"`, which will prevents iframe embedding, mitigating clickjacking [17].
- `SECURE_CONTENT_TYPE_NOSNIFF = True`, which will prevents MIME-type sniffing [18].
- Lastly, the Web server (NGINX) will be configured with Strict-Transport-Security (HSTS) max-age of 31536000 seconds (1 year) to make sure all connections are using HTTPS in the network level [19].

6.9 DEBUG Mode and Secret Key Management

DEBUG = False is set in production. In debug mode, Django exposes stack traces and settings values that may provide information leak [10]. SECRET_KEY is stored as an environment variable on the server and loaded via `os.environ.get("SECRET_KEY")` [10]. It is never committed to version control. The .env file is listed in .gitignore.

References

- [1] “Standards and Guidelines on digital accessibility | Australian Human Rights Commission.” Accessed: Apr. 05, 2026. [Online]. Available: <https://humanrights.gov.au/resource-hub/by-resource-type/guidelines-and-standards/guides-and-standards-disability-rights/chapter-3-standards-and-guidelines-digital-accessibility>
- [2] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.1.” Accessed: Apr. 05, 2026. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [3] “<label>: The Label element - HTML | MDN,” MDN Web Docs. Accessed: Apr. 05, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/label>
- [4] “Working with forms | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/5.0/topics/forms/>
- [5] “ARIA live regions - ARIA | MDN,” MDN Web Docs. Accessed: Apr. 05, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Guides/Live_regions
- [6] W. W. A. Initiative (WAI), “Cognitive Accessibility at W3C,” Web Accessibility Initiative (WAI). Accessed: Apr. 05, 2026. [Online]. Available: <https://www.w3.org/WAI/cognitive/>
- [7] “IBM Accessibility Toolkit.” Accessed: Apr. 05, 2026. [Online]. Available: <https://www.ibm.com/able/toolkit/www.ibm.com/able/requirements/requirements>
- [8] “Security in Django | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/5.0/topics/security/>
- [9] “Password management in Django | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/5.0/topics/auth/passwords/>
- [10] “Settings | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/6.0/ref/settings/>

- [11] “How to use sessions | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/6.0/topics/http/sessions/>
- [12] “3. Usage — django-axes 0.1.dev1+gb14b78a16 documentation.” Accessed: Apr. 05, 2026. [Online]. Available: https://django-axes.readthedocs.io/en/latest/3_usage.html
- [13] “Using the Django authentication system | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/5.0/topics/auth/default/>
- [14] “Cross Site Request Forgery protection | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/5.0/ref/csrf/>
- [15] “Security in Django | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/6.0/topics/security/>
- [16] “Performing raw SQL queries | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/6.0/topics/db/sql/>
- [17] “Clickjacking Protection | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/6.0/ref/clickjacking/>
- [18] “Middleware | Django documentation,” Django Project. Accessed: Apr. 05, 2026. [Online]. Available: <https://docs.djangoproject.com/en/6.0/ref/middleware/>
- [19] “HTTP Strict Transport Security (HSTS) and NGINX – NGINX Community Blog.” Accessed: Apr. 05, 2026. [Online]. Available: <https://blog.nginx.org/blog/http-strict-transport-security-hsts-and-nginx>

Use of Generative AI and Other Tools

Tool	Prompt (Main Goal)	Section Affected	What I Learned
Claude (claude.ai)	Refine the project overview, problem domain, and target audience. Claude also help me draft the list of key features for GeoMetaAssist	Section 1: Project Overview & Key Features	How to frame a SaaS product description clearly, separating problem domain, audience, and feature scope.
Claude (claude.ai)	Help to critically assess the normalization, integrate with the Django user entity, rationale of the entity and relationships, and also making sure my design follow the STAC specification.	Section 2: Database Design	DBML syntax, AbstractUser field inheritance in Django, BCNF design principles, and how to model SaaS subscription tiers relationally.
Claude Code	Initialise a Django project and help me build HTML mockups templates for all pages using Bootstrap 5 with realistic placeholder content (mock data).	Section 3: HTML Mockups	Django project and app structure, Bootstrap 5 grid and component usage. Claude Code was used heavily to assist with CSS and responsive layout implementation.
Claude (claude.ai)	Client-side (Turf.js) vs server-side (GeoPandas) GeoJSON metadata extraction across relevant criteria and produce a justified decision for each.	Section 4: Technology Research	The technical and architectural trade-offs between client-side and server-side extraction.
Claude (claude.ai)	Criticize the accessibility section, help me assess what 10 most relevant criteria for GeoMetaAssist, and help me map each to a specific UI component with relevant accessibility consideration.	Section 5: Accessibility	WCAG 2.1 structure, the POUR principles, and how specific criteria such as ARIA live regions, skip links, and colour contrast apply to a real application interface.
Claude (claude.ai)	Criticize my draft on security key points that I made mostly based on Django docs, and give ideas on what other security consideration other than what Django provides.	Section 6: Security	Distinction between role-level and object-level permission, HSTS and its role in preventing SSL stripping attacks, and why returning a 404 instead of a 403 on unauthorised object access avoids confirming resource existence to an attacker.

Note: the prompts listed above represent the main goal of each interaction, not the complete prompt sequence. Each section involved multiple follow-up exchanges to refine, correct, and adjust the output before it was incorporated into the design document.